

Bi-Weekly Research Progress Report

Project: IsoShard Proxy: A Distributed CP-Mode Transaction Coordinator & SQL Optimizer on AWS
Proponent: Sarthak Mushrif
Reporting Period: Weeks 5 & 6
Hours Logged: 20 Hours

1. Executive Summary of Progress

During the third reporting period, development shifted from the compiler frontend (AST parsing) to the middle-tier routing logic and network infrastructure. The primary achievement was the implementation of a Consistent Hashing algorithm designed to map parsed SQL queries to their designated database shards dynamically. Additionally, the foundational Java Netty TCP server was successfully bootstrapped, establishing the non-blocking I/O architecture required for high-throughput database proxying.

2. Detailed Task Breakdown (20 Hours)

- **Consistent Hashing Implementation (6 Hours):** Developed the core routing engine utilizing the MurmurHash3 algorithm. This component takes the routing key extracted by the AST Visitor (developed in Week 4) and deterministically calculates a 32-bit hash to map the query to a specific target shard.
- **Virtual Node Routing Table (4 Hours):** Implemented an in-memory routing table using a `TreeMap` (Sorted Map) to simulate the AWS RDS cluster topology. Integrated "Virtual Nodes" to ensure uniform data distribution across the shards and prevent data skewing on a single physical node.
- **Java Netty Server Bootstrapping (8 Hours):** Initialized the `ServerBootstrap` using Java Netty. Configured the `EventLoopGroup` (Boss and Worker threads) to handle incoming TCP connections asynchronously. This represents the birth of the actual proxy server that will listen for client SQL requests.
- **AST-to-Router Integration (2 Hours):** Wrote integration logic to pipe the output of the `IsoShardQueryVisitor` directly into the `ShardRouter`, successfully connecting the CS571 parser logic to the distributed systems routing logic.

3. Challenges & Roadblocks

- **Data Skew in Hashing:** Initial testing of the hashing algorithm revealed that a small number of physical shards (e.g., 3 nodes) led to uneven key distribution. This was resolved by implementing "Virtual Nodes," where each physical AWS RDS instance is represented by multiple logical points on the hash ring.
- **Netty Buffer Management:** Encountered minor memory leaks during early Netty server testing due to unreleased `ByteBuf` allocations. Implemented strict `ReferenceCountUtil.release()` protocols in the pipeline handlers to ensure garbage collection of network packets.

4. Objectives for Next Reporting Period (Weeks 7 & 8)

- Develop the custom Binary Codec (Encoder/Decoder) for the Netty pipeline to parse incoming raw byte streams into UTF-8 SQL strings.

- Implement the HikariCP connection pool to establish persistent, reusable JDBC connections from the Proxy to local/dummy database instances.
- Begin drafting the Phase 1 (Voting) logic of the Two-Phase Commit (2PC) protocol.

Appendix A: Technical Artifact - Consistent Hashing Router

The following Java class demonstrates the Consistent Hashing logic utilizing a Sorted Map and Virtual Nodes to distribute SQL queries across the simulated database cluster.

```
1 import java.util.SortedMap;
2 import java.util.TreeMap;
3 import com.google.common.hash.HashFunction;
4 import com.google.common.hash.Hashing;
5 import java.nio.charset.StandardCharsets;
6
7 public class ShardRouter {
8
9     private final HashFunction hashFunction = Hashing.murmur3_32_fixed();
10    private final SortedMap<Integer, String> hashRing = new TreeMap<>();
11    private final int VIRTUAL_NODES = 100;
12
13    // Initialize the cluster topology
14    public void addPhysicalShard(String rdsEndpoint) {
15        for (int i = 0; i < VIRTUAL_NODES; i++) {
16            String virtualNodeName = rdsEndpoint + "-VN" + i;
17            int hash = hashFunction.hashString(virtualNodeName,
18            StandardCharsets.UTF_8).asInt();
19            hashRing.put(hash, rdsEndpoint);
20        }
21    }
22
23    // Map a parsed AST routing key to a specific physical shard
24    public String routeQuery(String routingKey) {
25        if (hashRing.isEmpty()) {
26            throw new IllegalStateException("Cluster topology is empty. No
27            shards available.");
28        }
29
30        int queryHash = hashFunction.hashString(routingKey, StandardCharsets.
31        UTF_8).asInt();
32
33        // Find the next node on the consistent hash ring
34        SortedMap<Integer, String> tailMap = hashRing.tailMap(queryHash);
35
36        int nodeHash = tailMap.isEmpty() ? hashRing.firstKey() : tailMap.
37        firstKey();
38        return hashRing.get(nodeHash);
39    }
40 }
```

Listing 1: ShardRouter.java